

replicate- predictions- downloader

*A field guide to the archive run: how your tool walks
Replicate's ledger, sends out four porters, and leaves a
bookmark for next time.*

THE PROMISE

By the last exercise you should be able to explain how this tool works, step by step, to an artist who uses it or an engineer pairing with you on it, in your own words and without opening the code.

REPOSITORY
**closestfriend/replicate-predictions-
downloader**

SOURCE
index.js, 746 lines

VERSION TOURED
2.1.0 · 45 commits

TESTS
14, all passing

PRINTED
29 September 2026

SITTING
45–60 minutes, one pen

How to use this booklet

00

A walk through the code you wrote, in the order it runs. Read the guide once, then do the exercises without flipping back. The four **Explain how** exercises are the point: each gives you a listener and a budget and asks what the code *does*. Draft, self-check, redraft, then the key.

THE GUIDE

01	What You Built	2
02	The Map, and where things live	3
03	Follow One Run: one weekly --last-run	4
04	The Parts: what goes in, what comes out	7
05	Proper Names	8
06	Loose Ends, and where you'd go next	8

THE WORK

07	Field Exercises (8)	9
----	---------------------------	---

BEHIND THE STOP SIGN

08	Answer Key	13
09	Talking Points (read after you've drafted)	14
10	Passport, and Bring It Back	15

The key gives **ingredients**, not the dish: what a good answer contains, never a paragraph to memorize. In Loose Ends, **CONFIRMED** means reproduced or plainly visible in the code, and **SUSPECTED** means reasoned but not reproduced.

What You Built

01

You built a command-line tool that copies everything you've made on Replicate onto your own disk. It asks Replicate's API for your prediction history, one page at a time, saves the whole history as a JSON file, then downloads every output file of every successful prediction into a folder per model and zips each folder. Run it with `--last-run` and it only fetches what's new since the last complete run.

Replicate deletes data from API-made predictions after about an hour; website predictions are kept. Your tool is the copy that stays.

THE STACK, IN PLAIN ENGLISH

One Node.js file (`index.js`), run with `npx replicate-downloader`. Three dependencies: `commander` reads the flags, `dotenv` loads your token from a `.env` file, `archiver` makes the zips. Everything else, including the HTTP calls, uses what Node 18+ already has: `fetch` and streams.

The metaphor: the archive run. You send an archivist into a gallery that throws work away. They copy the **ledger** page by page and write an **index** of everything in it. Four **porters** carry the pieces home into **drawers**, one per model, and box each drawer. On the way out, the archivist leaves a **bookmark** so next week's run knows where to start.

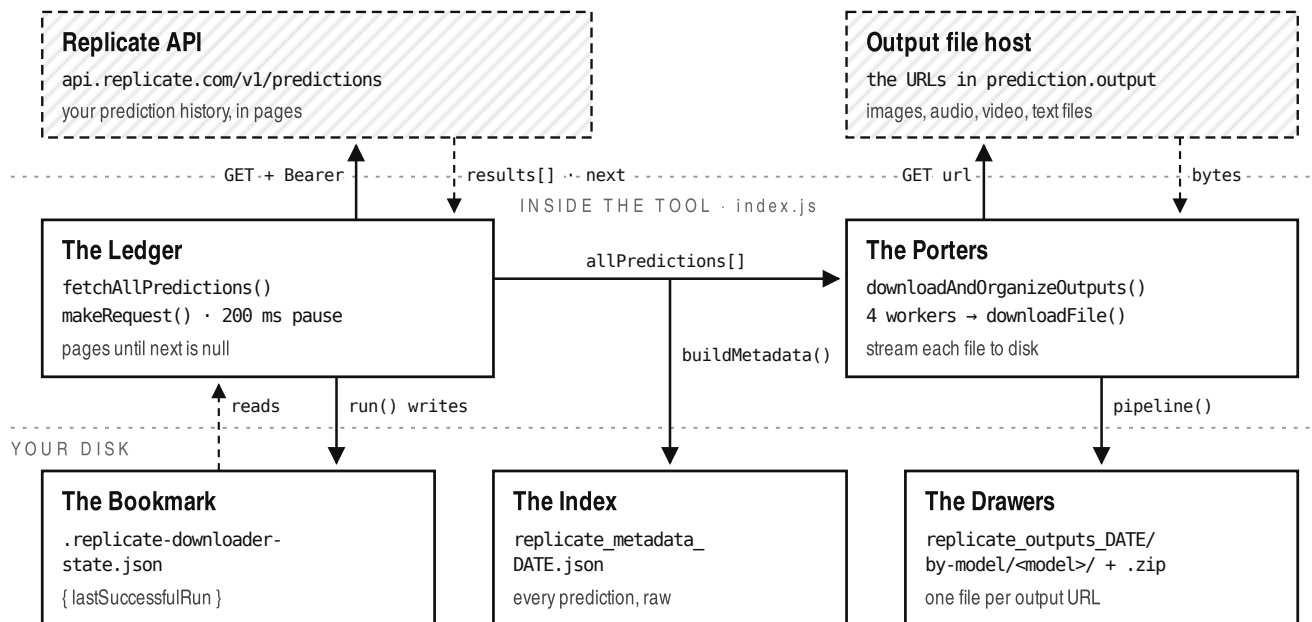
Where the metaphor breaks: a bookmark marks a page; yours marks a *moment*, the time the last complete run started. Nothing records which files you already have.

The Map

02

Hatched boxes are outside the tool. Solid arrows are requests or writes; dashed arrows are replies or reads. Every path is real.

OUTSIDE (NOT YOURS)



Where things live

THING	WHERE	LASTS
Your API token	<code>REPLICATE_API_TOKEN</code> , from the shell or <code>.env</code>	Until you change it. Never written anywhere by the tool
The list of predictions	<code>this.allPredictions</code> , in memory	One run
The index	<code>replicate_metadata_YYYY-MM-DD.json</code> in the folder you ran it from	On disk. A second run the same day overwrites it
The files	<code>replicate_outputs_YYYY-MM-DD/by-model/<model>/</code> and <code><model>.zip</code> beside each folder	On disk
The bookmark	<code>.replicate-downloader-state.json</code> , same folder	Until the next complete run replaces it
Knobs (delay, concurrency, zips)	The <code>CONFIG</code> object at the top of <code>index.js</code>	Change them by editing the file

All of these land in your *current directory*, so where you run the command is where the archive goes. `.gitignore` already excludes the outputs, the index and the bookmark.

Follow One Run

03

It's Sunday. You open a terminal in your archive folder and run `npx replicate-downloader --last-run`. Last Sunday's run finished cleanly. Since then you made one image set on the website: two images from `flux-dev`, prompt *"a lighthouse in fog, risograph"*. Here is everything that happens.

1 INDEX.JS · MAIN()

Read the token and the flags

Before `main()` runs, `dotenv.config()` copies `.env` into `process.env`, and `commander` parses your flags into `options`. `main()` checks three things: the token is set (otherwise it prints how to set it and stops), `--last-run` isn't combined with `--since` or `--until`, and `--source` is `web` or `api`. Then it builds `new ReplicateDownloader(token, { lastRun: true })` and calls `run()`.

2 GETDATEFILTERPARAMS() · LOADSTATE() · PARSEDATE()

Open the bookmark, turn it into a query

Because `lastRun` is set, the ledger reads `.replicate-downloader-state.json` and finds `lastSuccessfulRun: "2026-09-20T18:04:11.532Z"`. `parseDate` normalises it to ISO. Then one second is subtracted:

```
const sinceBuffer = new Date(new Date(parsedSince).getTime() - 1000);
params.append('created_after', sinceBuffer.toISOString());
```

The result is a query string: `created_after=2026-09-20T18:04:10.532Z`. That second of overlap means a prediction created at exactly the bookmark's instant is fetched, not lost.

`getDateFilterBounds()` computes the same bound again, for the check in hop 4.

3 FETCHALLPREDICTIONS() · MAKEREQUEST()

Copy the ledger, page by page

First it stamps `this.fetchStartTime` with the current time. That becomes next week's bookmark. Then it loops (simplified):

```
while (nextUrl) {
  const response = await this.makeRequest(nextUrl);
  this.allPredictions.push(...response.results);
  nextUrl = response.next; // a cursor URL, or null on the last page
  await new Promise(r => setTimeout(r, CONFIG.requestDelay)); // 200 ms
}
```

`makeRequest` sends `Authorization: Bearer <token>` and a `User-Agent` with the version. A non-OK status becomes a thrown `Error` carrying Replicate's `detail` message. Three details sit around that core loop. If the `next` URL has lost the date filter, the filter is appended again. If every prediction on a page is older than the bound, it stops early. And if any request throws, it logs the error and `breaks`, leaving `fetchComplete` false. This week there is one page, with your one prediction.

4 FETCHALLPREDICTIONS(), AFTER THE LOOP

Check the dates again, on your side

The server has already filtered by date, but the ledger filters once more: anything with `created_at` before the bound or after `--until` is dropped. A prediction with no `created_at` is kept. Dates are compared as ISO strings, which sort the same way as the times they name.

5

SAVEENHANCEDMETADATA() · BUILDMETADATA()

Write the index

Every prediction is grouped by model name. `extractModelName` takes the last part of `prediction.model` (`black-forest-labs/flux-dev` → `flux-dev`), falling back to `version.model`, then the `urls.get` path, then `unknown-model`. Statuses are counted under their real API names. The file looks like this:

```
{ "exported_at": "2026-09-27T17:30:02.118Z", "total_predictions": 1,
  "model_stats": { "flux-dev": { "total": 1, "succeeded": 1, "failed": 0, "canceled":
0 } },
  "predictions_by_model": { "flux-dev": [ { "id": "q7x2m4ab...", "prompt_summary":
    "a-lighthouse-in-fog-risograph", "has_output": true, ... } ] },
  "raw_predictions": [ ...every field Replicate returned... ] }
```

Failed and canceled predictions are in here too, and so is the output of models that return *text* rather than file URLs. The index is the only place that text is saved.

6

DOWNLOADANDORGANIZEOUTPUTS() · EXTRACTOUTPUTURLS()

Turn predictions into a job list

Only predictions with `status === 'succeeded'` and an `output` go forward. For each one, the model's folder is created, then `extractOutputUrls` walks the output, whatever its shape:

```
const collect = (value) => {
  if (typeof value === 'string') { if (value.startsWith('http')) urls.push(value);
}
  else if (Array.isArray(value)) value.forEach(collect);
  else if (value && typeof value === 'object')
Object.values(value).forEach(collect);
};
```

Your output is `{ images: [url1, url2] }`, so you get two jobs: `{ url, prediction, modelName: 'flux-dev', modelDir, fileIndex: 0 }` and the same with `fileIndex: 1`.

7

THE WORKER POOL · GENERATEFILENAME() · DOWNLOADFILE()

Four porters, one shared list

The pool is four async functions reading the same counter (simplified):

```
const worker = async () => {
  while (nextJob < jobs.length) {
    const job = jobs[nextJob++];
    try { await this.downloadFile(job.url, job.modelDir, baseName); ... }
    catch (e) { totalErrors++; } // log it, take the next job
  }
};
await Promise.all(Array.from({ length: Math.min(4, jobs.length) }, worker));
```

Each worker waits for its own download before taking another job, so at most four files are ever in flight. Taking a job (`jobs[nextJob++]`) happens with no `await` in the middle, and Node runs one piece of JavaScript at a time, so two workers can never take the same job. Here only two workers start, because there are only two jobs.

The name comes from `generateFilename`: date, model, prompt slug, the first eight characters of the ID, and `_2` for the second file. `2026-09-24_flux-dev_a-lighthouse-in-fog-risograph_q7x2m4ab`. The extension is added in `downloadFile`: from the URL if it has a sane one, else from the `Content-Type` header (`image/jpeg` → `jpg`), else `bin`. The body is streamed to disk with `pipeline()`, counting bytes as they pass. A bad status throws before any file is opened. A stream failure deletes the half-written file.

8

CREATEMODELZIP() · RUN()

Box the drawers, leave the bookmark

Each model folder is zipped with `archiver` at level 9, as `by-model/flux-dev.zip`. `run()` prints a summary (files, errors, size per model, status counts). Last, it writes the bookmark, but only under two conditions:

```
if (this.fetchComplete && successful > 0) {
  this.saveState({ lastSuccessfulRun: this.fetchStartTime, ... });
}
```

The new bookmark is the moment hop 3 *started*. Anything you create while the run is going is newer than that, so next Sunday picks it up. And if the ledger broke off halfway, the old bookmark stays, so next Sunday re-covers the gap.

The Parts

04

The whole tool is one class, `ReplicateDownloader`, plus a small CLI wrapper, in `index.js`. Here are its methods in the order a run uses them, by what goes in and what comes out.

METHOD	IN → OUT · AND WHAT TO CHECK WHEN IT MISBEHAVES
<code>main()</code>	flags and env → a configured downloader, or a printed error. <i>Check here when a flag is rejected or the token isn't found.</i>
<code>loadState()</code> / <code>saveState()</code>	the bookmark file → <code>{ lastSuccessfulRun }</code> . Read errors become a warning and an empty state.
<code>parseDate()</code>	"yesterday", "3 weeks ago", or anything new <code>Date()</code> accepts → ISO string. A month is 30 days, a year 365. <i>Check here when a date range is off by a day.</i>
<code>getDateFilterParams()</code> / <code>getDateFilterBounds()</code>	options + bookmark → the query string for the server, and the same bounds for the client-side check.
<code>makeRequest()</code>	URL → parsed JSON, or a thrown error with status and <code>detail</code> .
<code>fetchAllPredictions()</code>	→ <code>this.allPredictions</code> , <code>fetchStartTime</code> , <code>fetchComplete</code> . <i>Check here when a run fetches too much, too little, or stops early.</i>
<code>extractModelName()</code> · <code>extractPromptInfo()</code> · <code>sanitizeFilename()</code>	prediction → <code>flux-dev</code> · first prompt-like input → a lowercase, hyphenated slug of at most 50 characters (or <code>no-prompt</code>).
<code>buildMetadata()</code>	<code>allPredictions</code> → the index object, grouped by model with status counts.
<code>extractOutputUrls()</code>	any output shape → a flat list of <code>http...</code> strings. <i>Check here when a prediction has output but no file appears.</i>
<code>downloadAndOrganizeOutputs()</code>	successful predictions → folders, jobs, the four-worker pool, zips, counts.
<code>downloadFile()</code>	URL + folder + base name → <code>{ filepath, size }</code> , or a thrown error and no file. <i>Check here for wrong extensions or failed downloads.</i>
<code>run()</code>	the whole sequence, the summary, and the bookmark write.

Around it

- `test/unit.test.js`: 14 tests with `node:test`. Most start a tiny local HTTP server that plays Replicate, point `baseUrl` at it, and check real behaviour: Bearer auth, the watermark, at most four downloads in flight. Run with `npm test`.
- `example.js`: five ways to use the class as a library, each with its `run()` commented out.
- `package.json`: two command names (`replicate-downloader`, `replicate-predictions-downloader`) pointing at `index.js`, and Node 18 or newer.
- Outside the tool: Replicate's API, the output host (plain HTTPS, no token sent), and a Python port in a separate repo.

Proper Names

05

You built these by instinct. Here is what other engineers call them.

WHAT YOUR CODE DOES	IT'S CALLED	WHERE
Follows the <code>next</code> URL until it's null	Cursor pagination	<code>fetchAllPredictions</code>
Remembers <i>when</i> it last finished, not <i>what</i> it has; one second of overlap	High-water mark for an incremental sync , with an overlap window	the bookmark
Filters on the server, then checks again locally	Defense in depth (belt and braces)	hops 2 and 4
Four workers pulling from one list	Worker pool with bounded concurrency	<code>downloadAndOrganizeOutputs</code>
Writes bytes as they arrive, never the whole file in memory	Streaming ; <code>pipeline()</code> handles backpressure	<code>downloadFile</code>
Prompt → lowercase, hyphens, 50 characters	A slug	<code>sanitizeFilename</code>
Only moves the bookmark after a complete fetch	Commit only on success	<code>run()</code>

Loose Ends

06

What the code does today that you'll probably want to change. None of it loses data.

- **CONFIRMED** **A date-only `--until` leaves out that day.** `"2024-01-31"` parses to midnight at the *start* of the 31st (UTC), so the README's January example stops at the 30th. Fix: when `--until` has no time, add a day.
- **CONFIRMED** **Text outputs get an empty drawer.** A language model's output is a list of strings, not URLs, so no files are downloaded. The model's folder is still created, and zipped empty. The text is safe in the index.
- **CONFIRMED** **Settings the code never reads.** `.env.example` lists five options (`REQUEST_DELAY` and others), and `CONFIG.enhancedMetadata` exists, but only the token is read from the environment. The README also draws the zips one level higher than where they land.
- **CONFIRMED** **Exit code 0 on a missing token or a crash.** Scripts running it on a schedule can't tell those apart from success. A run whose fetch fails outright *does* exit 1.

Where you'd go next

- Save text outputs as `.txt` → a branch in `downloadAndOrganizeOutputs` for predictions whose `extractOutputUrls` list is empty.
- Skip files already on disk → a check before `downloadFile` in the worker.
- Retries → around `await this.downloadFile(...)` in the worker. From here, `explain-diff` covers each change.

Field Exercises

07

Eight exercises. The first four have right answers. The last four are **Explain how**, friendliest listener first. Draft, self-check, redraft, and only then the key.

01

FIND IT

THE DRAWERS

An output was saved as `..._q7x2m4ab.bin`. Which function chose `.bin`, what two things did it try first, and which function built the rest of the name?

02

PREDICT IT

THE LEDGER

You run `--since "2 days ago"` at exactly noon UTC on 10 March. What query parameter and value go to the API, and what happens to a page where every prediction is older than that?

03

PREDICT IT

THE BOOKMARK

A `--last-run` fetch gets pages 1 and 2, then page 3 returns a 500. What gets downloaded, what happens to the bookmark, and where does next week's run start?

04

TRACE IT

THE PORTERS

Ten jobs, four workers. Job 6's URL returns 404. Trace what happens to job 6, to the worker that took it, and to the other jobs.

05 **EXPLAIN HOW**

THE DRAWERS

LISTENER An artist who uses Replicate's website · **BUDGET** Three sentences

“If I run this tonight, what exactly ends up on my computer, and how are the files named?”

DRAFT 1

SELF-CHECK BEFORE DRAFT 2

- ☐ Said what happens, in order, not only what it's for
- ☐ Used the proper name, if one exists (p. 8)

- ☐ Circled every hedge (“kind of,” “basically,” “sort of”)
- ☐ Pitched at *this* listener

DRAFT 2 — BEFORE YOU TURN TO THE KEY

06 **EXPLAIN HOW**

THE BOOKMARK

LISTENER A friend who'll run it every week · **BUDGET** Four sentences

“How does it know what's new since last time? And what if my wifi drops halfway?”

DRAFT 1

SELF-CHECK BEFORE DRAFT 2

- ☐ Said what happens, in order, not only what it's for
- ☐ Used the proper name, if one exists (p. 8)

- ☐ Circled every hedge (“kind of,” “basically,” “sort of”)
- ☐ Pitched at *this* listener

DRAFT 2 — BEFORE YOU TURN TO THE KEY

07 **EXPLAIN HOW**

THE LEDGER

LISTENER An engineer pairing with you · **BUDGET** Four sentences

“Walk me through how you get the full list of predictions, with date filters, from the API.”

DRAFT 1

SELF-CHECK BEFORE DRAFT 2

- | | |
|--|--|
| ☐ Said what happens, in order, not only what it's for | ☐ Circled every hedge (“kind of,” “basically,” “sort of”) |
| ☐ Used the proper name, if one exists (p. 8) | ☐ Pitched at <i>this</i> listener |

DRAFT 2 — BEFORE YOU TURN TO THE KEY

08 **EXPLAIN HOW**

THE PORTERS

LISTENER An engineer pairing with you · **BUDGET** Four sentences

“How do downloads run in parallel without firing off hundreds of requests at once?”

DRAFT 1

SELF-CHECK BEFORE DRAFT 2

- | | |
|--|--|
| ☐ Said what happens, in order, not only what it's for | ☐ Circled every hedge (“kind of,” “basically,” “sort of”) |
| ☐ Used the proper name, if one exists (p. 8) | ☐ Pitched at <i>this</i> listener |

DRAFT 2 — BEFORE YOU TURN TO THE KEY

STOP.

Finish both drafts of every Explain-how exercise first. The key tells you what a good answer contains, not how to say it. Your words are the ones worth keeping.

FOLD OR TEAR HERE · ANSWER KEY & TALKING POINTS BEHIND

Answer Key

08

Stamp rule. Explain-how answers: Draft 2 hits every must-hit point (your wording, any order) and none of the traps. The first four: the “stamp if” line.

01 · Find it THE DRAWERS

ANSWER `downloadFile()` picks the extension: first `extensionFromUrl()` (the URL's last path segment), then `extensionFromContentType()` (the response header), and `bin` only if both give nothing (e.g. `application/octet-stream`). The base name comes from `generateFilename()`.

STAMP IF Both fallbacks, in order, and `generateFilename`.

02 · Predict it THE LEDGER

ANSWER `created_after=2026-03-08T11:59:59.000Z` (two days back, minus the one-second overlap; the exact milliseconds depend on the clock). If a whole page is older than the bound, pagination stops early and the client-side filter drops those predictions.

STAMP IF The parameter name, the minus-one-second, and the early stop.

03 · Predict it THE BOOKMARK

ANSWER The loop logs the error and breaks; `fetchComplete` stays false. Predictions from pages 1–2 are still indexed and downloaded. The bookmark is **not** written, so next week starts from the *old* bookmark and fetches the gap again (re-downloading pages 1–2 is harmless).

STAMP IF Partial download, bookmark untouched, next run starts from the old mark.

04 · Trace it THE PORTERS

ANSWER The worker's `downloadFile` sees `!response.ok` and throws before opening any file. The worker's `catch` logs it and adds to `totalErrors`, then its `while` loop takes the next job. The other workers never notice. `Promise.all` resolves when all four run out of jobs; the summary shows 9 downloaded, 1 error.

STAMP IF No file written, error counted, the same worker keeps going.

05 · Explain how THE DRAWERS

MUST HIT

- A folder named with today's date, with a subfolder (and a zip) for each model.
- Every file from every successful prediction, named `date_model_prompt_id`.
- One JSON file listing every prediction, including failed ones and text answers.
- It lands in whatever folder you ran it from.

TRAPS “It backs up your account” (it's predictions only). Leaving out the JSON, which is where text outputs live.

ALTITUDE No API words. One example filename beats a description of the pattern.

06 · Explain how THE BOOKMARK

MUST HIT	• <code>--last-run</code> reads a small file holding one timestamp. • It asks Replicate only for predictions created after that time (minus a second). • The timestamp saved is when the run <i>started</i>, so things you make mid-run are caught next time. • It's only saved if the whole list came through, so a dropped connection means next time starts from the old mark.
PROPER NAMES	High-water mark · incremental sync.
TRAPS	“It remembers which files it downloaded” (it stores a time, not a list). “It saves when it finishes” (it saves the start time).
ALTITUDE	Say “a timestamp in a file,” not <code>.replicate-downloader-state.json</code> .

07 · Explain how THE LEDGER

MUST HIT	• <code>GET /v1/predictions</code> with a Bearer token; dates go in as <code>created_after</code> / <code>created_before</code>, plus <code>source</code>. • Follow the <code>next</code> cursor until it's null, re-adding the filters if the cursor dropped them, with a 200 ms pause. • Stop early if a whole page is older than the bound, then re-filter locally. • Any error stops paging and marks the fetch incomplete.
PROPER NAMES	Cursor pagination · defense in depth.
TRAPS	“Page numbers” (it's a cursor URL). Skipping what happens on an error.
ALTITUDE	They'll want parameter names. Say them.

08 · Explain how THE PORTERS

MUST HIT	• Build a flat list of jobs, one per output URL. • Start four async workers; each takes the next job from a shared index and awaits it before taking another, so at most four are in flight. • Taking a job has no <code>await</code> in it, so on Node's single thread two workers can't take the same one. • Each file streams to disk; a failure is counted and the worker moves on. <code>Promise.all</code> waits for all four.
PROPER NAMES	Worker pool · bounded concurrency · streaming.
TRAPS	“Promise.all on every download” (that's unbounded). “Threads.”
ALTITUDE	Mention the test that proves the limit if they ask how you know.

Talking Points

09

For when you're stuck, not for reciting. Where your Draft 2 says it better, cross mine out.

ONE SENTENCE

I wrote a command-line tool that copies your whole Replicate history, every image, clip and prompt, onto your own disk, and on later runs only fetches what's new.

THIRTY SECONDS · NON-TECHNICAL

Replicate is where a lot of people run AI image and audio models, and it doesn't keep everything forever. My tool asks it for your whole history, saves a list of everything you've made, then downloads every file into a folder per model with names built from the date and the prompt. Run it again next week and it picks up where it left off.

TWO MINUTES · TECHNICAL LISTENER

It's a single Node file with three dependencies. It pages through Replicate's predictions endpoint by following the `next` cursor, with date filters sent as `created_after` and `created_before` and re-checked locally. It writes the history to a JSON index, then flattens every successful output into a list of download jobs. Four async workers pull from that list and stream each file to disk, taking the extension from the URL or the content type. For incremental runs it saves a high-water mark: the time the last *complete* fetch started. If a fetch breaks off, the mark isn't moved, so nothing is skipped.

Questions you'll get asked

“Does it need my password?”

No. It uses an API token you create on Replicate, read from an environment variable or a `.env` file, and never written anywhere.

“Will it download the same thing twice?”

With `--last-run`, only across an interrupted run, on purpose. Without it, it fetches everything each time.

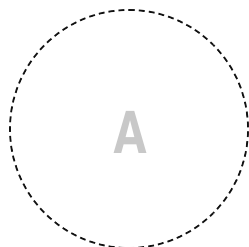
“What doesn't it do yet?”

Text outputs live only in the JSON index, and there's no retry for a failed download. Both are small changes in one function.

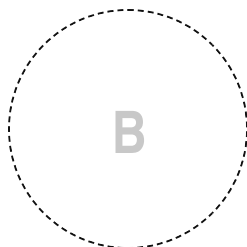
Passport

10

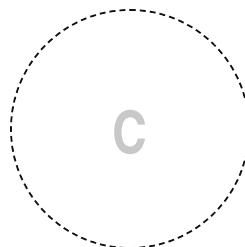
Stamp a district when an exercise in it meets the stamp rule. Be strict: Grade Mode will check you.



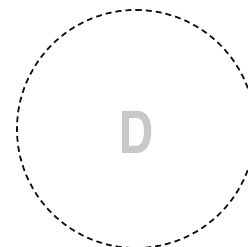
The Drawers
Ex. 01 · 05



The Ledger
Ex. 02 · 07



The Bookmark
Ex. 03 · 06



The Porters
Ex. 04 · 08

Bring it back

Photograph every exercise page, both drafts, flat and in daylight. Open a chat, invoke `grand-tour grade` and attach the photos. It reads your handwriting (and says what it couldn't read), grades each Draft 2 on **accurate · fitted · confident**, checks your stamps, then builds a talking-points card from *your* sentences wherever they passed.

Paper travels. This booklet contains no tokens or secrets; the API token appears only by its variable name.